

2.1 Architectural Style

FreshLens adopts a layered, service-oriented architecture that separates presentation, application, data, and background processing responsibilities into independent components. The architecture consists of two client applications—a vendor mobile application developed using Expo React Native and a platform administrator web application developed using Next.js—which communicate with a FastAPI backend through RESTful APIs over HTTPS.

The backend coordinates authentication, business logic, data persistence, asynchronous task processing, and machine learning inference while maintaining strict tenant isolation through PostgreSQL Row-Level Security (RLS). Long-running freshness classification tasks are executed asynchronously using Celery workers connected through Redis, allowing API requests to return immediately without waiting for machine learning inference.

This architectural approach improves modularity, maintainability, scalability, and separation of concerns while supporting the functional and non-functional requirements defined in the Software Requirements Specification.

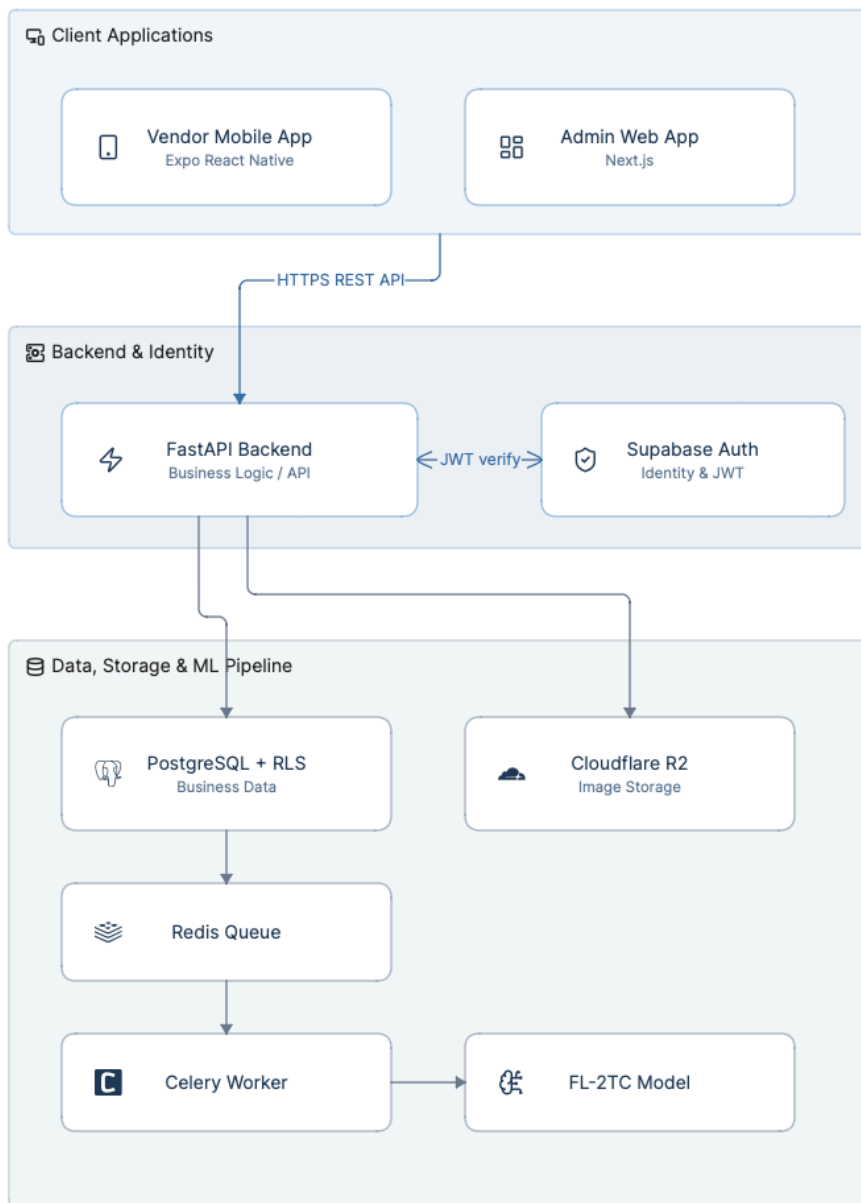
2.2 High-Level System Architecture

The below diagram illustrates the overall architecture of the FreshLens system.

Vendors interact with the mobile application to authenticate, capture produce images, submit scans, and view freshness results. Platform administrators use the web application to manage vendors, products, and platform-wide information.

Both client applications communicate with the FastAPI backend through REST APIs. Authentication is handled by Supabase Auth using JWT access tokens. Business data is stored in PostgreSQL with Row-Level Security to enforce tenant isolation.

Captured images are stored in Cloudflare R2. When a scan is submitted, the backend creates an asynchronous processing task using Redis and Celery. The FL-2TC machine learning model processes the image in the background and stores the classification results in PostgreSQL. The mobile application subsequently retrieves the completed results through the API.



2.3 Major System Components

Component	Responsibility
Vendor Mobile App	Allows vendors to authenticate, capture produce images, submit scans, and view dashboards and alerts.
Platform Admin Web	Enables administrators to manage vendors, products, and platform information.
FastAPI Backend	Provides REST APIs, business logic, authentication integration, and request coordination.
Supabase Auth	Authenticates users and issues JWT access tokens.
PostgreSQL	Stores tenant data, scan results, alerts, and inventory records.
Cloudflare R2	Stores uploaded produce images.
Redis	Maintains asynchronous task queues.
Celery Worker	Executes background freshness classification tasks.
FL-2TC Model	Performs produce classification and freshness prediction.

2.4 Architectural Views

The FreshLens software architecture is represented using multiple architectural views. Each view describes the system from a different perspective to provide a complete understanding of the system design.

- **Use-Case View:** Describes how vendors and platform administrators interact with the FreshLens system and the major functions available to each user.
- **Logical View:** Describes the major software components, their responsibilities, and the relationships between them.
- **Process View:** Describes the runtime behaviour of the system, including API requests, asynchronous scan processing, and communication between components.
- **Deployment View:** Describes the physical deployment of software components across client devices, servers, databases, and cloud services.
- **Implementation View:** Describes the organization of the source code, modules, packages, and project structure used to implement the system.
- **Data View:** Describes the major data entities, database schema, and relationships used to store and manage system data.